

# Sistema de módulos do NodeJS

## Objetivo da Aula

Conhecer o sistema de módulos do NodeJS, identificar quais deles pertencem ao núcleo ou são de terceiros e criar módulos de desenvolvedor.

## Apresentação

A plataforma NodeJS utiliza o sistema de módulos, que são conjuntos de elementos de software plugáveis, sempre voltados para alguma funcionalidade específica. Diversos desses módulos já são oferecidos no núcleo da plataforma, visando a operações comuns, como o acesso ao sistema de arquivos ou a criação de clientes e servidores HTTP.

Veremos que o sistema de módulos, além de garantir uma boa organização, traz grande expansibilidade para a plataforma, pois, além de podermos definir nossos próprios módulos, podemos utilizar módulos de terceiros. Com base na comunicação com o GitHub, o comando npm permite acrescentar esses módulos de terceiros, transformando tarefas complexas na simples escolha da ferramenta correta para o projeto.

## 1. Fundamentos do Sistema de Módulos

Certamente, você já sentiu a necessidade de utilizar uma mesma função em diversos programas diferentes. Para tal, diferentes plataformas trabalham com o conceito de bibliotecas, um elemento básico para reuso.

No ambiente do NodeJS, esse reuso é garantido pelo sistema de módulos, os quais podem ser classificados em três tipos:

- Módulos do núcleo (Core);
- Módulos de terceiros; e
- Módulos do desenvolvedor.

Os módulos do núcleo são instalados com a distribuição básica do NodeJS, englobando elementos que lidam com o sistema operacional e a comunicação HTTP, entre outros, enquanto os módulos de terceiros permitem a inclusão de novas funcionalidades. Quanto aos módulos do desenvolvedor, são conjuntos de funções e classes utilizados ao longo de todo o projeto, aumentando o reuso e facilitando a manutenção ao evitar que você replique o código.

Você pode criar um módulo de funções definindo um arquivo do tipo **JS** e adotando o elemento **exports** antes de cada função pública.

```
// Arquivo circuloF.js
const { PI } = Math;
exports.area = (raio) => PI * raio ** 2;
exports.perimetro = (raio) => 2 * PI * raio;
```

Aqui, temos duas funções sendo exportadas: uma para cálculo da área de um círculo, e outra para o perímetro, além da importação do valor de **PI** a partir de **Math**. Tendo utilizado o nome **circuloF.js** para o arquivo, você pode importá-lo e utilizar suas funções, como no exemplo seguinte.

```
// Arquivo index.js
const c1 = require("./circuloF.js");
console.log(`Um círculo de raio 1 tem área ${c1.area(1)} `+
`e perímetro ${c1.perimetro(1)}`);
```

Você também pode adotar uma abordagem orientada a objetos, criando uma classe para o círculo, como no exemplo seguinte.

```
// Arquivo circuloC.js
const { PI } = Math;
module.exports = class Circulo {
  constructor(raio) {
    this.raio = raio;
  }
}
```

```
area = () => PI * this.raio ** 2;
perimetro = () => 2 * PI * this.raio;
};
```

Com a definição de uma classe, a forma de utilização é modificada. Tendo adotado o nome **circuloC.js** para o arquivo, você pode importá-lo e utilizar sua classe, como no exemplo seguinte.

```
// Arquivo index.js
const Circulo = require("./circuloC.js");
const c1 = new Circulo(1);
console.log(`Um círculo de raio 1 tem área ${c1.area()} `+
`e perímetro ${c1.perimetro()}`);
```

A importação pode ser simplificada, modificando a linha com **require** para o comando **import**.

```
import { Circulo } from "./circuloC.js";
```

Para que o comando import funcione, você deve transformar seu projeto em um módulo, acrescentando **type**, com valor **module**, no arquivo **package.json**.

```
{
  "name": "exemplo02",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
```

```
"license": "ISC",  
"type": "module"  
}
```

Também é necessário mudar a definição da classe, no arquivo **circuloC.js**, para utilizar o comando **export**.

```
export class Circulo {
```

Você pode criar seus módulos e utilizá-los em diversos projetos, fazendo a importação com o comando **npm install**. Para experimentar essa possibilidade, crie a pasta **calculadora**, no mesmo nível do projeto atual, e abra no **VS Code**.

Com o diretório aberto, crie o arquivo **package.json**, utilizando o conteúdo da listagem seguinte, para configurar o módulo.

```
{  
  "name": "calculadora",  
  "main": "./principal.js",  
  "exports": {  
    ".": "./principal.js"  
  },  
  "type": "module"  
}
```

Segundo o que foi especificado, o nome do módulo é **calculadora**, e o ponto de entrada é **principal.js**, sendo exportado o mesmo arquivo como padrão. Em seguida, o arquivo definido como ponto de entrada deve ser criado, utilizando a listagem apresentada a seguir.

```
export default class Calculadora {  
  somar = (x,y) => x+y;  
  subtrair = (x,y) => x-y;  
}
```

Aqui, foi criada uma classe **calculadora**, com os métodos **somar** e **subtrair**, sendo exportada como padrão, devido ao uso de **export default**. Em seguida, volte para o projeto principal e utilize o comando seguinte.

```
npm install ../Calculadora
```

Você poderá observar que foi criado o diretório **node\_modules**, no qual existirá uma cópia do módulo **calculadora**, além de algumas alterações efetuadas no arquivo **package.json**, com a inclusão da dependência. É dessa forma que o NodeJS gerencia a inclusão de módulos externos no projeto, podendo buscar em um diretório local ou nos repositórios do GitHub.

Você já pode alterar o arquivo **index.js** para utilizar o módulo **calculadora**.

```
import Calculadora from "calculadora";
const c1 = new Calculadora();
console.log(c1.somar(1,2));
```

Para remover o módulo do projeto, basta executar o comando **npm uninstall**, fornecendo o nome do módulo que será removido.

```
npm uninstall calculadora
```



### Você Sabia?

Para diminuir o tamanho do backup de seu projeto, você pode apagar o diretório **node\_modules**, no qual ficam todas as bibliotecas adicionadas, pois ele é reconstruído com o comando **npm install**.

## 2. Módulos do Núcleo

Diversas bibliotecas são disponibilizadas no NodeJS, de forma nativa, sendo conhecidas como core modules ou **módulos do núcleo**. Outras ferramentas e bibliotecas podem ser instaladas **globalmente** utilizando o comando **npm install** com a opção **g**.

No núcleo básico, você encontrará os módulos necessários para gerenciar as conexões em rede, acessar o sistema de arquivos e recuperar informações do sistema operacional, entre diversas outras funcionalidades essenciais. Eles são os únicos que poderão ser utilizados sem adicionar ao projeto, bastando efetuar a importação dos arquivos JavaScript.

A tabela seguinte descreve os principais módulos nativos.

**Tabela 1:** Alguns pacotes do núcleo do NodeJS

| Módulo | Descrição                                                        |
|--------|------------------------------------------------------------------|
| http   | Gerencia o protocolo HTTP para criação de clientes e servidores. |
| fs     | Acessa o sistema de arquivos.                                    |
| path   | Gerencia caminhos para os arquivos.                              |
| url    | Gerencia endereços na web.                                       |
| os     | Fornece informações do sistema operacional.                      |

*Fonte: Elaboração própria.*

Você pode utilizar o módulo **http** para definir um servidor de forma simples, como na listagem apresentada a seguir.

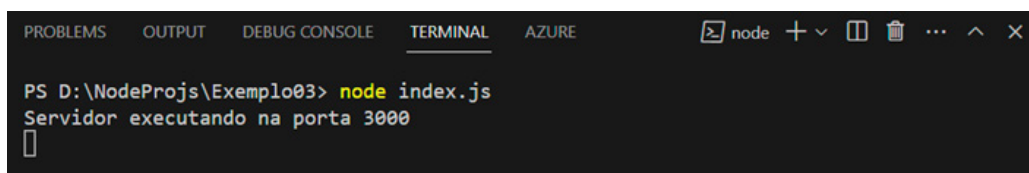
```
const $http = require("http");
const $os = require("os");
$http.createServer((req, res)=>{
  res.writeHead(200, {"Content-Type": "text/html"});
  res.write("<html><body>");
  res.write("<h1>Servidor</h1>");
  res.write(`Plataforma: ${$os.platform}<br/>`);
  res.write(`Endereco: ${$os.hostname}<br/>`);
  res.write("</body></html>");
  res.end();
}).listen(3000);
console.log("Servidor executando na porta 3000");
```

Além do módulo `http`, foi utilizado o módulo **os**, para identificar a plataforma e o endereço do servidor, através dos atributos **platform** e **hostname**. Note que temos uma arrow function com os parâmetros **req**, para a requisição, e **res**, para a resposta, permitindo construir uma página HTML de retorno, além de emitir o código 200, indicando sucesso, e o tipo de conteúdo.

Após a construção do servidor, pelo método **createServer**, seu método `listen` é invocado para a porta 3000, gerando um thread que ficará escutando qualquer requisição efetuada até que o programa seja encerrado. Por estar executando de forma paralela, não ocorre bloqueio, e a mensagem informando acerca da execução é exibida em seguida.

Como nos demais exemplos, utilize **node index.js** para executar.

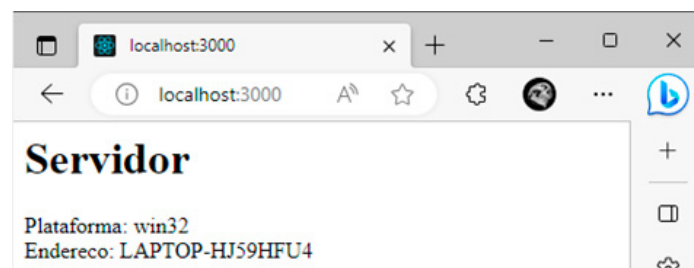
**Figura 1:** Execução do servidor na porta 3000.



Fonte: Elaboração própria.

Com o servidor em execução, escolha o navegador de sua preferência e acesse o endereço **http://localhost:3000** para testar seu aplicativo. Estando tudo correto, você deverá ter um retorno como o que é apresentado a seguir.

**Figura 2:** Acesso ao servidor via navegador



Fonte: Elaboração própria

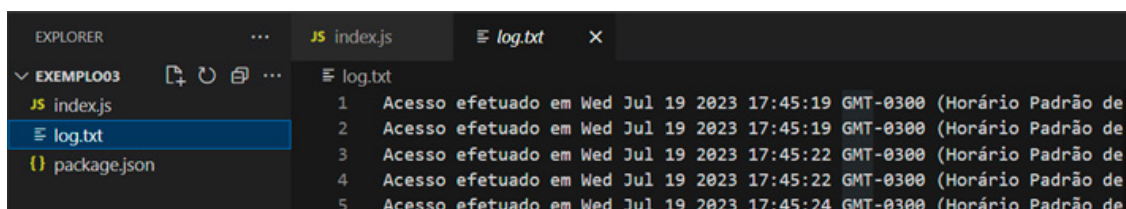
O servidor poderia gravar um log de acesso, utilizando a biblioteca **fs**, como no código alterado de **index.js**, apresentado a seguir.

```
const $http = require("http");
const $os = require("os");
```

```
const $fs = require("fs");
const savelog = () => {
  $fs.appendFileSync('log.txt',
    `Acesso efetuado em ${new Date()}\n`);
}
$http.createServer((req, res)=>{
  res.writeHead(200, {"Content-Type": "text/html"});
  res.write("<html><body>");
  res.write("<h1>Servidor</h1>");
  res.write(`Plataforma: ${$os.platform}<br/>`);
  res.write(`Endereco: ${$os.hostname}<br/>`);
  res.write("</body></html>");
  res.end();
  savelog();
}).listen(3000);
console.log("Servidor executando na porta 3000");
```

Esse é um exemplo de adição de conteúdo em arquivo texto, utilizando modo de escrita síncrono, apenas para verificar o funcionamento da biblioteca fs. Após efetuar a modificação, execute novamente o servidor e verifique como o acesso via navegador causa a criação e atualização do arquivo de log no projeto.

**Figura 3:** Arquivo de log gerado pelo servidor



*Fonte: Elaboração própria.*



### 3. Módulos de Terceiros

Grande parte do poder e da flexibilidade do NodeJS reside na quantidade notável de módulos disponíveis na web. Através de uma integração direta com o GitHub, o comando **npm** permite instalar bibliotecas para os mais diversos fins, como a criação de servidores minimalistas com **express** e o acesso a diversos bancos de dados com **sequelize**.

Um procedimento muito comum nos sistemas web é a geração de relatórios no formato PDF. Você pode fazer isso com as bibliotecas **express** e **PdfKit**. Crie um diretório para o projeto, abra no VS Code e digite os comandos seguintes.

```
npm init
npm install express
npm install pdfkit
```

No passo seguinte, crie o arquivo **index.js**, utilizando o seguinte conteúdo.

```
const express = require('express');
const ArquivoPDF = require('pdfkit');
const app = express();
app.get('/gerarPDF', (req, res) => {
  const doc = new ArquivoPDF({bufferPages: true});
  let dados = [];
  doc.on('data', dados.push.bind(dados));
  doc.on('end', () => {
    let pdf = Buffer.concat(dados);
    res.writeHead(200, {
      'Content-Length': Buffer.byteLength(pdf),
      'Content-Type': 'application/pdf'}).end(pdf);
  });
  doc.font('Times-Roman').fontSize(16)
    .text('APENAS UM EXEMPLO');
```

```
doc.end();  
});  
app.listen(3000);
```

No código, temos a criação de um servidor express, escutando na porta 3000, que responde no modo **GET**, para a rota **gerarPDF**, utilizando conteúdo em PDF. Note que o cabeçalho deve incluir a quantidade de bytes da saída e o tipo de conteúdo, como **application/pdf**.

Para a geração do PDF, foi utilizada a biblioteca PdfKit, apoiada em um vetor de trabalho com o nome **dados**. Dois eventos são programados para o gerador de PDF, em que **'data'** associa o vetor de trabalho e **'end'** recupera o conteúdo PDF, emitindo para o navegador através de **'res'**.

Ao final do tratamento da requisição, temos a inclusão do texto, com tipo de fonte e tamanho especificados, o que irá ativar o evento data, bem como a chamada para end de PdfKit, que completa a criação do conteúdo PDF, ativando o evento de mesmo nome e completando o processo.

Executando o servidor e acessando **http://localhost:3000/gerarPDF**, você terá como resultado a abertura do PDF no seu navegador. Esse é apenas um pequeno exemplo do poder das bibliotecas oferecidas via npm, pois, apesar de um código simples, temos diversas tarefas complexas sendo implementadas.

Outra necessidade comum na construção de sites é a geração dinâmica de **QR code**, como em processos de autenticação, por exemplo. Basicamente, um código desse tipo é definido pela transformação de um texto em uma imagem que o representa ou código Scalable Vector Graphics (**SVG**).

Utilize o comando seguinte para acrescentar a biblioteca **QRCode**.

```
npm install qrcode
```

Em seguida, acrescente uma segunda rota ao código do servidor, antes da linha com o comando **listen**, através do código seguinte. Todas as rotas devem ser definidas antes que o servidor passe a escutar a rede.

```
app.get("/qrcode", (req, res) => {  
  QRCode.toString('https://www.google.com', {  
    errorCorrectionLevel: 'H', width: 200, type: 'svg'
```

```

}, (err, data) => {
  if (err) throw err;
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write("<html><body>");
  res.write(data);
  res.write("</body></html>");
  res.end();
});
})

```

A biblioteca permite gerar o QR code tanto para imagem quanto para texto SVG, bastando fornecer o texto que estará encapsulado, além de atributos como largura e tipo de saída. Para gerar na forma de texto, é utilizado o método **toString**, que executa de forma assíncrona, retornando o resultado para uma **callback**, aqui no formato arrow function. Note que o texto codificado, no parâmetro **data**, será utilizado diretamente no código HTML retornado para o navegador.

Executando o servidor e acessando **http://localhost:3000/qrcode**, você terá a saída apresentada a seguir.

**Figura 4:** Saída para a rota QR code



*Fonte: Elaboração própria.*

Apontando a câmera do celular para o código gerado, o endereço fornecido como texto é recuperado de forma automática, e você pode navegar diretamente para o site do Google.

## Considerações Finais da Aula

Após conhecermos os tipos de módulo que existem para o NodeJS, vimos como utilizá-los, através das diretivas `require` e `import`. Também, pudemos ver como exportar funcionalidades de um arquivo JavaScript, com base no elemento `module` e na cláusula `export`.

Com os conhecimentos adquiridos, fomos capazes de criar nosso próprio módulo de desenvolvedor, importando, em seguida, para outro projeto, através do comando `npm`. Trata-se de uma abordagem que permite dividir o sistema em componentes menores e reutilizáveis, garantindo a construção de bibliotecas, o que poderá trazer aumento de produtividade em projetos subsequentes.

Em um terceiro passo, utilizamos bibliotecas de terceiros, importadas para o projeto através do `npm`, diminuindo nosso esforço para incluir funcionalidades complexas em nossos sistemas. Como exemplo, fomos capazes de utilizar PDF na resposta ao usuário e gerar QR codes – tarefas que exigiriam grande esforço e conhecimento sem o uso dos módulos importados.

## Materiais Complementares

### Artigo

#### **How to create and read QR codes in Node.js**

2023, Kayode Adeniyi.

O artigo cobre todos os passos necessários para gerar QR codes no ambiente do NodeJS, tanto para gravação em arquivo quanto para apresentação em sites.

Link para acesso: <https://blog.logrocket.com/create-read-qr-codes-node-js/> (acesso em 12 set. 2023.)

### Artigo

#### **Requisitando módulos no Node.js: tudo o que você precisa saber**

2022, Samer Buna. Tradução de Fernando Mota Freitas.

Artigo sobre a criação e o uso de módulos do NodeJS. Faz parte do livro *NodeJS Beyond the Basics* do autor Samer Buna.

Link para acesso: <https://abre.ai/gKJJ> (acesso em 12 set. 2023.)

## Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKFM>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKFN>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKFP>. Acesso em: 12 set. 2023.